TP ICAP - Software Engineering Competency (Back End)

Thank you for taking the time to do our software engineering "problem", the assessment consists of two parts:

- Your solution to the User Stories (see Problem Statement)
- Your answers to the Technical Questions

General Guidance

We understand that people have busy lives so to help you use your time wisely, and focus in the right areas we, we offer the following guidance:

- Our focus is on how you implement the User Stories and how "production-ready" your code is
- When we review assessments, we treat your code **and** your answer to Question 2 (Technical Questions) as your "complete solution".. so write the code you need to, and use the "answer" to state what else you would do to make your code "production-ready"
- Most candidates spend between 2 and 4 hours on the assessment
- Include a console application to demonstrate how your code would be used - hard coding the input data
- **DO NOT** spend time creating other mechanisms to input data (such as CSV/JSON/TEXT parsers, REST endpoints, etc.)
- **You must include tests**

Engineering Tips

- Use C#
- Use any frameworks, packages, libraries you like, as long as they don't require a license
- Your code should compile and run without additional work from us
- **You must include tests**

Submitting Your Assessment

In order to avoid bounced emails we would prefer you to upload your resulting zip file to an online file hosting service such as Google Drive, Drobox, etc. and share the URL. If you don't have access to such a service please talk to your Talent Acquisition contact at TP ICAP for alternative instructions.

- Create a single zip file named **{yourname}-{role-applied-for}.zip** containing:
    - Your solution to the Problem Statement
    - Your answers to the Technical Questions

Classification: Public

- Please reduce the size of the zip file by removing
    - BIN and OBJ folders
    - Automatically downloaded libraries such as NUGET packages
- Please remove anything that might be caught by virus scanning software
    - EXE and DLL files

Problem Statement

Matching “buy” and “sell” orders is at the heart of the TP ICAP business. Matching algorithms are usually implemented using advanced libraries, but for the sake of this assessment, we ask you to use your own code to implement the simple matching algorithms we've provided. We also recommend that you don’t solely focus on the algorithms but also consider how your solution might fit into a larger codebase.

The task is to create an application that accepts the “order book” as a parameter. The application should then, according to the algorithm, match the buy and sell orders and return the following details.

- CompanyId
- OrderId
- Direction
- Volume
- Notional
- MatchState (NoMatch, FullMatch, PartialMatch, InvalidOrder, etc.)
- Matches:
    - OrderId
    - Notional
    - Volume

Classification: Public

User Story - One

Given I am a **user running the application**, when I **submit an "order book" (containing orders) to be matched using the *price-time-priority* algorithm**, then I want to see **the state of the order book**.

Under a basic price-time-priority algorithm, the earliest active buy order at the highest price (notional) takes priority over any subsequent order at that price, which in turn takes priority over any active buy order at a lower price.

Order Book

CompanyId	OrderId	Direction	Volume	Notional	OrderDateTime
A	A1	Buy	100	4.99	09:27:43
B	B1	Buy	200	5.00	10:21:46
C	C1	Buy	150	5.00	10:26:18
D	D1	Sell	150	5.00	10:32:41
E	E1	Sell	100	5.00	10:33:07

Acceptance Criteria:

- A1 - NoMatch - the higher priced orders took priority
- B1 - FullMatch
    - D1 - 150
    - E1 - 50
- C1 - PartialMatch
    - E1 - 50
- D1 - FullMatch
    - B1 - 150
- E1 - FullMatch
    - B1 - 50
    - C1 - 50

Classification: Public

User Story - Two

Given I am a **user running the application**, when I **submit an "order book" (containing orders) to be matched using the *pro-rata* algorithm**, then I want to see **the state of the order book**.

Under a basic pro-rata algorithm, the system prioritizes active orders at a particular price (notional), proportional to the relative size of each order.

Order Book

CompanyId	OrderId	Direction	Volume	Notional	OrderDateTime
A	A1	Buy	50	5.00	09:27:43
B	B1	Buy	200	5.00	10:21:46
C	C1	Sell	200	5.00	10:26:18

- Total Buy = 250
- Buy 50 / Total Buy = 20%
- Buy 200 / Total Buy = 80%
- 20% * Sell 200 = 40
- 80% * Sell 200 = 160

Acceptance Criteria:

- A1 - PartialMatch
    - C1 - 40
- B1 - PartialMatch
    - C1 - 160
- C1 - FullMatch
    - A1 - 40
    - B1 - 160

Classification: Public

Technical Questions

Please provide your answers to the following in a markdown file named **answers.md**.

1. How much time did you spend on the engineering task?
2. What would you add to your solution if you'd had more time?
3. What do you think is the most useful feature added to the latest version of C#?
    a. Include a code snippet that shows how you've used it.
4. How would you track down a performance issue in production?
    a. Have you ever had to do this?

Thanks for your time, we look forward to hearing from you!

The TP ICAP team

Classification: Public